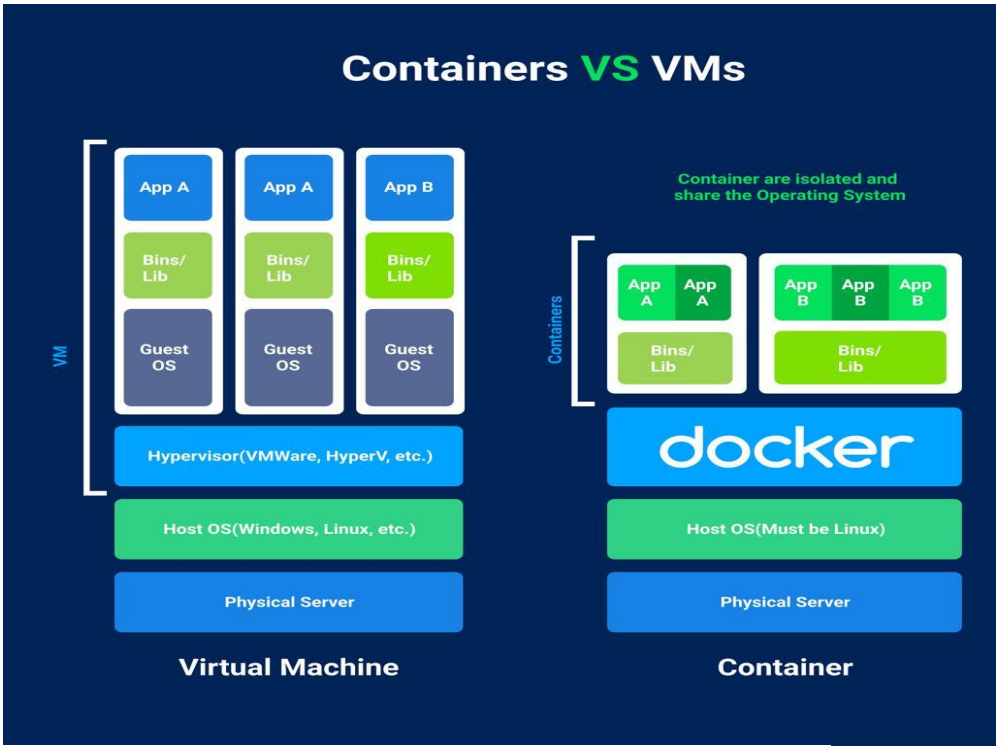
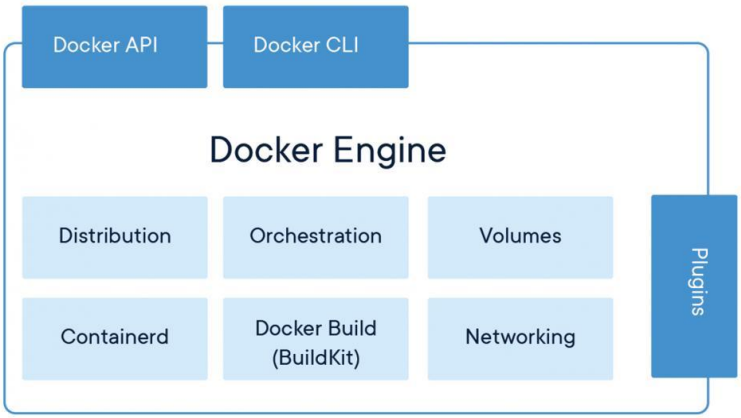
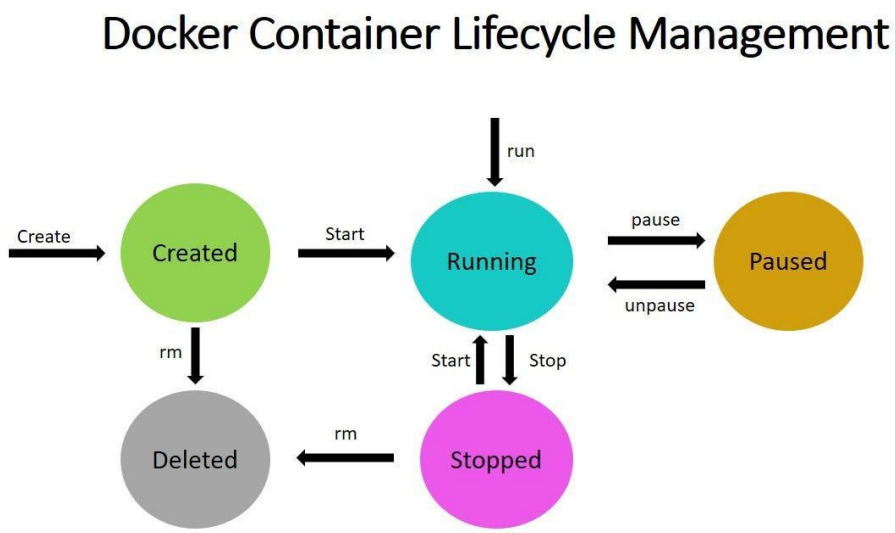
Docker

1. **What is Docker:**
   Docker is an **open-source containerization platform** used to build, package, and run applications inside **containers**.







7

A **container** packages:

- Application code
- Runtime
- Libraries
- Dependencies
- Configuration

So the application runs **the same way on any system**.

Example

If you create an application on your laptop, Docker ensures it runs **exactly the same on server, cloud, or another machine**.

---

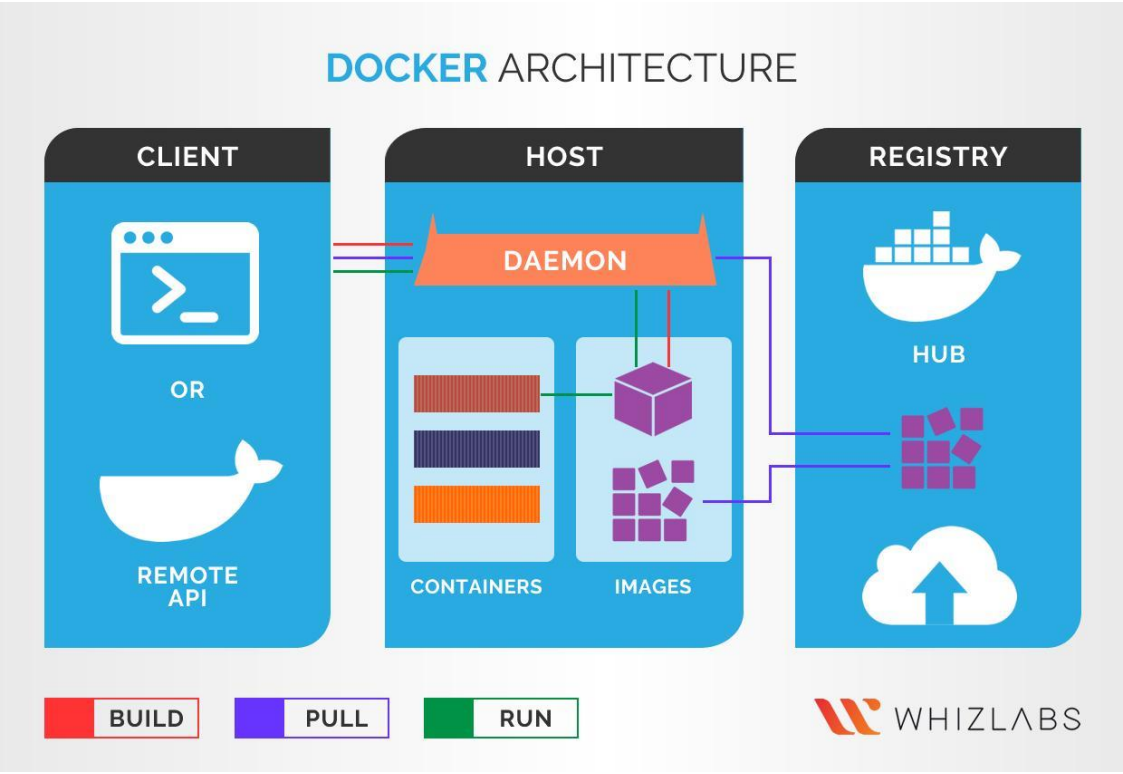
2. Virtual Machine vs Docker

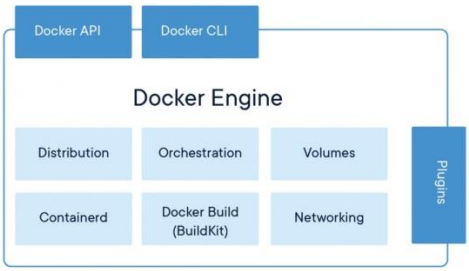
Feature	Virtual Machine	Docker
OS	Each VM has full OS	Uses host OS
Size	Large (GBs)	Small (MBs)
Boot time	Minutes	Seconds
Performance	Slower	Faster
Resource usage	High	Low

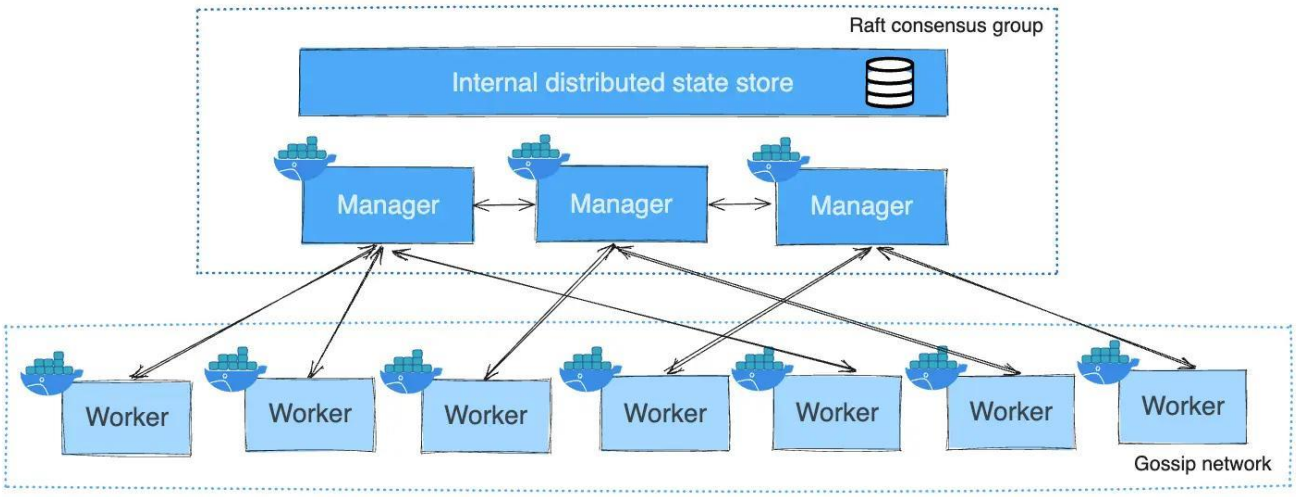
Conclusion:
Docker is **lightweight and faster than VMs**.

---

3. Docker Architecture







6

Docker uses **Client–Server architecture**.

Components

1. Docker Client

Command line tool used to interact with Docker.

Example:

```
docker build
docker pull
docker run
```

2. Docker Daemon

Background service that manages:

- containers

- images
- networks
- volumes

3. Docker Registry

Stores Docker images.

Example:

- Docker Hub
- AWS ECR
- Google Container Registry

---

4. Docker Key Components

1. Docker Image

A **read-only template** used to create containers.

Example:

```
nginx image
ubuntu image
mysql image
```

2. Docker Container

A **running instance of an image**.

Example:

```
docker run nginx
```

3. Dockerfile

A text file that contains instructions to build a Docker image.

Example:

```
FROM ubuntu
RUN apt update
RUN apt install nginx -y
CMD ["nginx","-g","daemon off;"]
```

4. Docker Hub

A public repository for Docker images.

Example:

```
docker pull nginx
```

---

5. Installing Docker (Linux)

For Amazon Linux / RHEL

```
sudo yum update -y
sudo yum install docker -y
```

Start Docker

```
sudo systemctl start docker
```

Enable Docker

```
sudo systemctl enable docker
```

Check version

```
docker --version
```

Add user to docker group

```
sudo usermod -aG docker ec2-user
```

---

6. Important Docker Commands

Check Docker Version

```
docker --version
```

Check Docker Information

```
docker info
```

Download Image

```
docker pull nginx
```

List Images

```
docker images
```

Run Container

```
docker run nginx
```

Run Container in Background

```
docker run -d nginx
```

Run Container with Port Mapping

```
docker run -d -p 8080:80 nginx
```

List Running Containers

```
docker ps
```

List All Containers

```
docker ps -a
```

Stop Container

```
docker stop containerID
```

Remove Container

```
docker rm containerID
```

Remove Image

```
docker rmi imageID
```

---

7. Docker Container Lifecycle

```
Create → Start → Run → Stop → Delete
```

Commands:

Create container

```
docker create nginx
```

Start container

```
docker start containerID
```

Stop container

```
docker stop containerID
```

Delete container

```
docker rm containerID
```

---

8. Dockerfile Example (Simple Application)

Example:

```
FROM ubuntu

RUN apt update

RUN apt install nginx -y

CMD ["nginx","-g","daemon off;"]
```

Build Image

```
docker build -t myimage .
```

Run Container

```
docker run -d -p 8080:80 myimage
```

---

9. Docker Volumes

Used to **store persistent data**.

Example:

```
docker volume create myvolume
```

Use volume

```
docker run -d -v myvolume:/data nginx
```

Benefits

- Data persistence
- Data backup
- Data sharing between containers

---

10. Docker Networking

Types of networks:

Network	Description
bridge	Default network
host	Container uses host network
none	No network
overlay	Used in Docker Swarm

Create network

```
docker network create mynetwork
```

Run container in network

```
docker run -d --network mynetwork nginx
```

---

11. Docker in DevOps

Docker is used in **CI/CD pipelines**.

Example workflow:

```
Developer Code
     ↓
GitHub
     ↓
Jenkins Build
     ↓
Docker Image Build
     ↓
Push to Docker Hub
     ↓
Deploy Container
```

Benefits:

- Fast deployment
- Consistent environments
- Easy scaling
- Microservices architecture

---

12. Docker Compose

Used to run **multiple containers together**.

Example: Web + Database

docker-compose.yml

```
version: "3"

services:

  web:
    image: nginx
    ports:
      - "8080:80"

  db:
    image: mysql
    environment:
      MYSQL_ROOT_PASSWORD: root
```

Run services

```
docker-compose up
```

Stop services

```
docker-compose down
```